

ML Specification Report — playground-series-s3e19

Forecasting Mini-Course Sales • AIDE agent (20-step tree search, Claude Sonnet via shim)

Figures grounded in the run's `facts_aide.json`, the agent's own `report.md` and `best_solution.py` (run `2-brown-falcon-of-pleasure`), its `journal.json` tree, and `config.yaml` for task metadata. Dataset shapes were measured directly from the run's own `input/` copy of the competition files.

Superseded run. This report documents AIDE run `2-brown-falcon-of-pleasure`, which the benchmark later **voided** for input contamination (see the provenance note under Data) and replaced with a clean re-run, `0-meaty-zebra-of-improvement` (local holdout SMAPE 4.3718, 19 scored / 1 buggy, 2,053.8 s of execution; submitted 2026-07-29 for public 48.39715 / private 48.34131). Every figure below describes the voided run — including its 11.1944 holdout and its 78.8 s compute total — and is kept only as a record of it. The benchmark's current s3e19 figures are the clean re-run's; `benchmark_results/rerun_three_way.csv` still records `winner_priv = nvidia` for this competition.

Contents	
Forecasting Mini-Course Sales • AIDE agent (20-step tree search, Claude Sonnet via shim)	1
Overview	1
Data	2
Models & Architecture	4
Training procedures	5
The search trajectory	5
Specification fields	7
Inference procedures	7
Evaluation & Benchmarking	8

Overview

The task is to forecast daily `num_sold` for 27,375 test rows covering the whole of 2022, given five years of history (2017–2021) across 5 countries × 3 stores × 5 products (metric: SMAPE, minimized). This is the only genuine **time-series extrapolation** problem in the batch, and AIDE treated it as one: over **20 steps** it abandoned direct row-level regression after the first draft and built a two-stage decomposition — forecast the country-day total, then split it into product-store rows by historical shares. **17 steps returned a score and 3 crashed**, improving from **holdout SMAPE 18.8816** to **11.1944** at **step 18**.

$$18.8816 - 11.1944 = 7.6872$$

That is by far the largest relative improvement AIDE achieved anywhere in this batch (s3e11, s3e14, s3e16, s3e19), and the modelling reasoning behind it is the most sophisticated of the four runs — it correctly identified that gradient-boosted trees cannot extrapolate beyond the training range and solved it with a **monotone constraint on year** rather than by bolting on a linear trend, and it correctly identified and corrected the **retransformation bias** that `expm1` introduces when a model is trained in log space.

And it still finished at the **37.7th percentile, rank 732/1174** — the worst of AIDE's four runs in this batch — with a private LB of **49.61325**, because the local number it optimized was worth almost nothing:

$$49.61325 / 11.1944 = 4.43$$

The private score is 4.4× the local estimate. Everything below is best read against that gap. AIDE built a defensible pipeline, validated it on a single 2021 holdout, and had no way to learn that a single holdout year does not tell you what 2022 will do. `facts_aide.json` records `local_winner = mine` and `lb_winner = nvidia`.

Why it matters. Hierarchical demand forecasting — predict the aggregate, then allocate it down a product/store hierarchy — is the standard shape of real retail planning, and the failure mode on display here (a validation protocol that flatters a model which cannot actually extrapolate) is the standard way such projects go wrong in production.

Data

Purpose of Data. Forecast `num_sold` for every (date, country, store, product) row of 2022 — a **regression / time-series forecasting** task scored by SMAPE, minimized (`config.yaml` :

```
evaluation_metric: smape, optimization_direction: minimize, target_column: num_sold ). Data
```

Format is **tabular CSV with a date key**. **Data Volume**, measured from the run's `input/` directory, is **136,950 training rows × 6 columns / 27,375 test rows × 5 columns** — a fully rectangular panel of **75 series** (5 countries × 3 stores × 5 products) observed daily, with train spanning **2017-01-01 to 2021-12-31** and test spanning **2022-01-01 to 2022-12-31**.

Data Quality is clean: **zero missing values, zero duplicate rows** once `id` is excluded, and no gaps in the panel. The target `num_sold` is a positive integer taking 1,028 distinct values over [2, 1380], with mean 165.5226 and skew 1.7474 — the most strongly right-skewed target in the batch, which justifies the log modelling AIDE applied and makes the retransformation-bias correction it later added genuinely necessary rather than cosmetic.

The defining property of the data is not distributional but **temporal**: the test period lies entirely outside the training range on the one axis (`year`) that carries the trend. A tree ensemble can only predict values

it has seen, so an unconstrained GBDT asked about 2022 will return its 2021 leaf values. AIDE understood this — its step-5 plan states that "tree-based total-sales models fail to extrapolate" — but it took until step 7 to find a fix that worked.

Two provenance facts belong on the record. First, the run's `input/` directory contains `train_processed.csv` (24 columns) and `test_processed.csv` (22 columns) in addition to the raw files, and **the champion reads the processed files, not the raw ones**. The calendar features it uses — cyclical sin/cos encodings of month, day-of-week and day-of-year, `is_weekend`, `is_month_start/end`, `is_new_year`, `quarter`, `weekofyear`, and integer codes `country_cat` / `store_cat` / `product_cat` — were therefore **staged into the input tree rather than engineered by AIDE**. They are deterministic functions of the date and leak nothing about the target, so the result stands, but AIDE's feature-engineering contribution on this competition is smaller than the pipeline's complexity suggests.

Second, and more concretely, **the champion's holiday-feature logic is a no-op**. The code scans for any column whose name contains `"holiday"` and appends it to the stage-1 feature list; no such column exists in `train_processed.csv`, so the list stays at exactly the 12 base features. The agent's `report.md` nevertheless claims that "dynamically detected 'holiday'-named columns were incorporated into the total-sales model." The journal settles it: **step 11 exists solely to add holiday signals and scored 13.2797 — bit-identical to its parent step 10 at 13.2797**. A step of the budget was spent adding nothing, and the identical metric is the proof.

Annotation Guidelines. The label is `num_sold`, a positive integer. Submissions are integer daily unit counts scored by SMAPE; the champion rounds its continuous product to integers and floors it at 1.

Feature Set. The champion uses a deliberately small stage-1 feature set and does all its remaining work outside the model:

Group	Features
Trend	<code>year</code> (monotone-increasing constrained)
Cyclical seasonality	<code>doy_sin</code> , <code>doy_cos</code> , <code>month_sin</code> , <code>month_cos</code> , <code>dow_sin</code> , <code>dow_cos</code>
Calendar flags	<code>is_weekend</code> , <code>is_month_start</code> , <code>is_month_end</code> , <code>is_new_year</code>
Entity	<code>country_cat</code> (native LightGBM categorical)
Detected holidays	(none found — the dynamic scan matches no column)

That is **12 features for stage 1**. Stage 2 uses no learned features at all: it is a lookup table of shares keyed on `(country, product, store, month)` with a `(product, store)` fallback. AIDE tested richer stage-1 features and they hurt — second- and third-order Fourier harmonics of day-of-year scored 13.7733 against a running best of 13.2797.

Splitting strategy. A **time-based holdout**: train on 2017–2020, validate on 2021, with the final model refit on all of 2017–2021 (reusing the holdout model's `best_iteration` as a fixed round count) before predicting 2022. Shares are likewise computed from ≤ 2020 for validation and from all years for the final fit, with the recency decay re-centred on the target year each time. This is the correct shape of protocol

for a forecasting task and AIDE deserves credit for rejecting random CV at step 0. What it does not do is test extrapolation across more than one horizon, and AIDE's own Future Work section names the omission: "Only a single train/validation split (2021 holdout) was used; rolling-origin or multiple holdout years would give more robust estimates of generalization to 2022." The run was **submitted to Kaggle**: public 48.99617, private 49.61325.

Models & Architecture

Purpose of Architecture. Forecast 2022 daily unit sales for 75 series, minimizing SMAPE. **Architecture Type** is a **two-stage hierarchical decomposition**: a single LightGBM regressor for the country-day aggregate, plus a deterministic **shrinkage-blended share allocator** that disaggregates each predicted total across the 15 product-store cells within that country. There is no ensemble, no stacking and no seed-bagging anywhere in the champion — it is one model and one lookup table.

Input Format for stage 1 is a 12-column matrix at **country-day granularity** — 5 countries $\times$ 1,826 days of history, a 9,130-row aggregate, which is why every node in this run executes in seconds. Stage 2 consumes the full row-level panel. **Input Dimension** is 12 features per aggregated row for the learned component.

Architecture Description.

- **Stage 1 — total forecast.** LightGBM, `objective="regression"` (L2) with `metric="mae"`, `learning_rate=0.05`, `num_leaves=31`, `seed=42`, up to 2,000 boosting rounds with 50-round early stopping against the 2021 holdout, `categorical_feature=["country_cat"]`, and — the decisive setting — `monotone_constraints` set to 1 for year and 0 for every other feature. The monotone constraint forces every split on year to be non-decreasing, which lets the ensemble project the upward trend past 2021 instead of flattening at its last observed leaf.
- **Retransformation correction.** Because stage 1 is fitted on `log1p(total)`, inverting with plain `expm1` under-predicts the mean by Jensen's inequality. The champion instead computes a **Duan smearing estimator** — `smear = mean(exp(residual))` over the in-sample training residuals — and returns `exp(pred_log) × smear - 1`, clipped at 0. A separate smearing factor is recomputed for the full-data refit.
- **Stage 2 — share allocation.** For each (country, product, store) cell, daily shares of the country total are computed and weighted by an exponential recency decay `DECAY = 0.6` per year of distance from the target year, so 2021 counts about 0.6 as much as the target year and 2017 about 0.6⁵. Two estimates are formed — a month-conditioned share on `(country, product, store, month)` and an unconditioned share on `(country, product, store)` — and combined by **shrinkage** with strength `K_SHRINK = 5.0`: `(w · share_month + 5 · share_overall) / (w + 5)`, where `w` is the decayed observation count backing the monthly estimate. The blend is renormalized to sum to 1 within each `(country, month)` group, with a global `(product, store)` share as fallback for unseen combinations.

Model Complexity is one **LightGBM model of at most 2,000 trees at 31 leaves**, plus a share table of $5 \times 5 \times 3 \times 12 = 900$ blended coefficients and two scalar hyperparameters (`DECAY` , `K_SHRINK`) that were never tuned — both are first-guess values fixed at the step where they were introduced. Relative to the other three runs in this batch, the learned capacity here is tiny; nearly all the pipeline's behaviour lives in hand-specified structure.

Training procedures

The search trajectory

The `journal.json` tree has **5 root drafts (steps 0, 1, 5, 6, 7)** and 15 descendant nodes; the champion sits at depth 4 along **7 → 10 → 14 → 16 → 18**. Three nodes crashed — one `ValueError` and two `KeyErrors`, all of them column-plumbing errors from merging the processed and raw files.

Step	Parent	What AIDE tried	Holdout SMAPE
0	—	LightGBM on row-level <code>log_num_sold</code> , all features (first draft)	18.8816
1	—	two-stage decomposition (draft)	<code>ValueError</code>
2	1	fix: deduplicate the <code>country_cat</code> column list	18.2163
3	2	day-of-week-conditioned shares	18.2164
4	2	explicit per-country linear trend + LGBM residual	21.175
5	—	decomposition variant (draft)	<code>KeyError</code>
6	—	per-group log-linear trend extrapolation (draft)	25.7345
7	—	monotone year constraint on the total model (draft)	13.2873
8	5	fix: recover <code>country</code> / <code>product</code> / <code>store</code> from the raw files	<code>KeyError</code>
9	8	fix: resolve <code>date_x</code> / <code>date_y</code> suffixing on merge	22.247
10	7	recency-weighted shares (exponential decay)	13.2797
11	10	add dynamically-detected holiday columns	13.2797
12	10	2nd/3rd Fourier harmonics of day-of-year	13.7733
13	10	global-total model + country share	13.4462
14	10	month-conditioned shares with hard fallback	11.5323
15	14	month + day-of-week conditioned shares	11.5809
16	14	shrinkage-blended month shares (continuous, not hard fallback)	11.5083
17	16	nested shrinkage, day-of-week blended into the month blend	11.5146
18	16	global smearing bias correction (champion)	11.1944
19	18	per-country smearing factors	11.4475

This is the cleanest search of the four runs in this batch in the sense that **every large gain is attributable to a specific, named, defensible idea**, and each was proposed by AIDE reading the prior nodes' scores.

The decomposition itself is worth 0.67:

$$18.8816 - 18.2163 = 0.6653$$

The **monotone year constraint at step 7** is the run's decisive move, and it is the more impressive for what it replaced: steps 4 and 6 had both tried to solve extrapolation by fitting explicit linear trends and regressing the residual, scoring 21.175 and 25.7345 — substantially worse than doing nothing. AIDE abandoned that family and put the constraint inside the model instead:

$$\begin{aligned} 18.2163 - 13.2873 &= 4.9290 \\ 4.9290 / 7.6872 &= 0.641 \end{aligned}$$

That single node is 64% of the run's whole improvement. **Month-conditioned shares at step 14** are the second lever, and they had to be discovered against a prior negative result — step 3's day-of-week conditioning had changed nothing (18.2163 → 18.2164), so there was little evidence that finer share keys would help:

$$13.2797 - 11.5323 = 1.7474$$

The **smearing correction at step 18** closes it out:

$$11.5083 - 11.1944 = 0.3139$$

The negative results are equally informative and AIDE read them correctly. Finer conditioning without regularization hurt (month+dow hard fallback, 11.5809, versus 11.5323 for month alone); the same conditioning with shrinkage helped slightly (11.5083); and over-segmenting the bias correction per country regressed it (11.4475 versus 11.1944), which AIDE attributed to per-country residual samples being too small to estimate separate factors — a correct diagnosis.

Three nodes were lost to a **dead debug branch**. Steps 5, 8 and 9 form a chain in which a decomposition variant crashes on missing `country / product / store` columns (they exist only in the raw files, not the processed ones), the repair crashes again on `date_x / date_y` merge suffixing, and the second repair finally runs — scoring **22.247**, worse than the step-0 baseline. Fifteen percent of the step budget went to recovering a branch that was never competitive:

$$3 / 20 = 0.15$$

Compute, by contrast, was trivial. Total execution time was **78.8 s** across 20 steps, mean 3.9 s, max 38.7 s; the champion node ran in **1.4 s**. The entire 20-step search cost less than a minute and a half of CPU, because stage 1 fits on a 9,130-row aggregate rather than the 136,950-row panel. Set against the s3e14 run in the same batch:

AIDE spent 172× less compute on the hardest problem in the batch than on the easiest, and it is the competition where it placed worst. Cheap nodes are not a virtue in themselves — here they meant the search could afford twenty experiments and still never test the one thing that mattered, which was whether the protocol generalized.

Specification fields

The **Loss Function** is L2 (`objective="regression"`) on `log1p(country-day total)` , monitored with MAE for early stopping — note that **neither is SMAPE**. The competition metric is never optimized directly anywhere in the pipeline; it is only used to score the holdout. AIDE's own Future Work names this: "a custom objective more directly aligned with SMAPE could better optimize the final evaluation metric." The **Optimization Algorithm** is gradient boosting with monotone-constrained splits — **not** SGD/ADAM. The **Learning Rate** is the boosting shrinkage, `0.05` . There is **no Learning Rate Scheduler** (N/A — convergence is governed by 50-round early stopping on the 2021 holdout; the final refit instead reuses the holdout model's `best_iteration` as a fixed round count) and **no Batch Size** (N/A — full-dataset boosting on a 9,130-row aggregate).

Training Duration is 78.8 s across the whole search, of which the champion accounts for 1.4 s. **Training Memory** was **not recorded** — no instrumentation, no cap; the aggregated stage-1 matrix is negligible in any case. **Transfer Learning** is N/A (no pretrained weights); AIDE's analogue is its **journal-conditioned prompt**, and it is visibly load-bearing on this run — step 7's plan explicitly contrasts itself with the failed explicit-trend nodes ("instead of an explicit linear trend..."), and step 16's plan cites step 15's regression as its reason for switching from hard fallback to shrinkage. **Data Augmentation** is N/A (tabular / time series); the analogue is the **recency-weighted share estimation**, which reweights history rather than adding to it.

Reproducibility Standards are adequate at the script level: `seed=42` for LightGBM, deterministic aggregation and share arithmetic, and no stochastic post-processing, so the champion re-runs as written. The search itself is not reproducible — node proposals come from LLM sampling — and there is no replay gate. One reproducibility subtlety is worth flagging: the final model's round count is inherited from the holdout model's `best_iteration` , so the submitted forecast depends on an early-stopping decision made against 2021 data that is then folded into the training set. That is standard practice for refit-on-all workflows, but it means the final model was never validated at the size it was actually trained to.

Inference procedures

Decision Threshold is **N/A** — SMAPE on a continuous-valued count target, so no classification boundary is involved. **Post-processing**, however, is substantial and consists of four ordered operations: (1) apply the smearing factor when inverting the log prediction, `exp(pred_log) × smear - 1` ; (2) clip the

predicted country total at 0; (3) multiply by the blended share to reconstruct each row; and (4) **round to the nearest integer and clip at a minimum of 1**, since `num_sold` is a positive count and SMAPE's denominator misbehaves near zero. Any row whose `(country, product, store, month)` key is missing falls back to the global `(product, store)` share, and any row still unmatched after the merge is filled with the median prediction before the integer cast.

Inference Duration and **Inference Memory** were **not recorded**; AIDE times whole scripts, not their scoring phase. Both are negligible here — stage 1 scores 1,825 country-day rows and stage 2 is a table join.

Evaluation & Benchmarking

Task for Performance Evaluation. Forecast `num_sold` for every 2022 row and minimize SMAPE.

Performance Metrics. The champion reaches **holdout SMAPE 11.1944** on 2021, and scores **48.99617 public / 49.61325 private**, placing **rank 732/1174, 37.7th percentile**. The scored trajectory was 18.8816 → 18.2163 → 13.2873 → 13.2797 → 11.5323 → 11.5083 → **11.1944**. The gap between that final local number and the leaderboard is the single most important figure in this report:

$$\begin{aligned} 49.61325 - 11.1944 &= 38.41885 \\ 49.61325 / 11.1944 &= 4.43 \end{aligned}$$

Public and private agree with each other and disagree with the holdout, which rules out leaderboard noise as the explanation:

$$49.61325 - 48.99617 = 0.61708$$

The diagnosis follows from the architecture. Predicting 2021 from 2017–2020 requires extrapolating **one** year; predicting 2022 from 2017–2021 also requires one year, but the model that does it was refit on a longer history at a round count chosen for the shorter one, and — more fundamentally — a monotone constraint only guarantees that the forecast will not decrease, not that it will increase by the right amount. The share table has the same exposure: 2022's product mix is estimated from a recency-weighted average of years the model has seen, with no mechanism to detect that the mix has moved. Every one of AIDE's 7.69 points of local improvement was measured on a single holdout year, and the leaderboard says most of it did not transfer.

Performance Benchmarking. All three agents attacked this competition independently; the private-LB figures below are the only cross-lane numbers used in this report. **Local scores are not comparable across lanes** — each agent chose its own validation protocol — so only leaderboard figures are tabulated.

Agent	Approach	Private LB SMAPE	Note
NVIDIA	reproduce-agent distilling a public kernel	48.26558	winner

Agent	Approach	Private LB SMAPE	Note
AIDE	20-step tree search → monotone total + shrinkage shares	49.61325	second
Our agent	from-scratch pipeline	50.45537	third

50.45537 – 49.61325 = 0.84212

49.61325 – 48.26558 = 1.34767

`facts_aide.json` records `local_winner = mine` and `lb_winner = nvidia`. AIDE lands in the middle: 0.84 ahead of our own agent, 1.35 behind NVIDIA, and all three cluster in a band that puts them well below the top of a 1,174-team leaderboard. That clustering is itself the finding — **this is the competition where all three approaches struggled, and where the percentile placement (37.7) is by far the worst of AIDE's four runs in this batch** (compare 91.4 on s3e16, 73.3 on s3e11, 71.2 on s3e14).

The honest summary is that AIDE's reasoning here was better than its result. The monotone-constraint idea, the shrinkage-blended share table and the smearing correction are all techniques a competent forecaster would endorse, and AIDE found them unaided, in sequence, from negative evidence. What it lacked was any instrument capable of telling it that its 11.19 was fictional. Its own Future Work section identifies the missing instrument — rolling-origin validation across multiple holdout years — but a 20-step budget with no submission feedback gives an agent no reason to spend steps on validating the validator, and every reason to spend them chasing the number it can see.